Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos – Sección 21

Docente – Kevin Antonio Velásquez Aguilar



Ejercicio 6

Estuardo Antonio Molina Donis - 26878

Ingeniería en Ciencia de la Computación y Tecnologías de la Información

08/11/2026

Requisitos funcionales

Información que debe guardar el sistema

1. El sistema debe permitir administrar las integraciones de APIs externas utilizadas por una plataforma de comercio electrónico.

2. Cada integración debe tener un identificador que permita reconocerla y buscarla dentro del sistema.

3. Cada integración debe guardar el nombre del proveedor al que pertenece.

4. El sistema debe manejar diferentes tipos de integraciones, como pagos, logística, facturación electrónica y consultas.

5. Cada integración debe guardar su estado de conexión para indicar si se encuentra conectada o desconectada.

6. Cada integración debe guardar su tiempo de latencia promedio en milisegundos.

7. El sistema debe permitir consultar los parámetros de configuración de una integración.

8. El programa debe comenzar con al menos diez integraciones de diferentes categorías.

9. Todas las integraciones deben almacenarse en una sola colección polimórfica.

10. El sistema debe permitir agregar nuevas integraciones sin modificar la lógica principal de las operaciones.

11. Las integraciones deben compartir características generales, pero podrán tener comportamientos diferentes según el servicio que representan.

Interfaces y operaciones

12. El sistema debe utilizar interfaces para definir los comportamientos que pueden realizar las integraciones.

13. La interfaz Transaccional permitirá representar operaciones financieras, como cobros y reintegros.

14. La interfaz Documentable permitirá representar operaciones relacionadas con la generación de documentos.

15. La interfaz Auditable permitirá representar operaciones relacionadas con el registro o consulta de información de auditoría.

16. Las integraciones de pagos podrán realizar las operaciones financieras que tengan permitidas.

17. Las integraciones de logística podrán consultar información relacionada con envíos y generar documentos cuando corresponda.

18. Las integraciones de facturación electrónica podrán generar documentos relacionados con FEL.

19. Las integraciones de consultas podrán revisar información sin necesidad de ejecutar cobros o reintegros.

20. Una integración no estará obligada a implementar operaciones que no correspondan a su función.

21. El sistema deberá identificar cuándo una operación no es compatible con una integración.

22. Cuando se solicite una operación no compatible, el sistema deberá ignorarla y mostrar un mensaje claro.

23. Una operación no compatible no deberá provocar que el programa termine inesperadamente.

24. El sistema debe permitir ejecutar diferentes comportamientos mediante polimorfismo vía interfaces.

25. El programa debe aprovechar la sobrecarga de métodos (overloading) cuando sea necesaria para realizar operaciones con diferentes parámetros.

Consultas y administración

26. El sistema debe permitir mostrar todas las integraciones registradas.

27. Al listar las integraciones, debe mostrar su identificador, proveedor, tipo, estado de conexión y latencia.

28. El sistema debe permitir buscar una integración mediante su identificador.

29. Cuando se encuentre una integración, debe mostrar sus parámetros de configuración.

30. Cuando no exista una integración con el identificador solicitado, debe mostrar un mensaje indicando que no se encontró.

31. El sistema debe permitir ordenar las integraciones por su tiempo de latencia promedio.

32. El ordenamiento debe utilizar la interfaz Comparable de Java.

33. El catálogo debe permitir identificar cuáles integraciones tienen mayor o menor latencia.

34. El sistema debe permitir consultar las integraciones conectadas y desconectadas.

35. El sistema debe permitir utilizar diferentes categorías de integraciones desde una misma colección.

Interfaz gráfica y funcionamiento

36. El programa debe aplicar el patrón Modelo-Vista-Controlador (MVC).

37. El modelo debe encargarse de almacenar los datos y realizar las operaciones de las integraciones.

38. La vista debe encargarse de mostrar la información y recibir las acciones del usuario.

39. El controlador debe coordinar la comunicación entre la vista y el modelo.

40. El programa debe permitir interactuar mediante un menú de consola.

41. Como funcionalidad adicional, el sistema tendrá una interfaz gráfica desarrollada con Java Swing.

42. La interfaz gráfica debe permitir consultar las integraciones mediante una tabla.

43. La interfaz gráfica debe incluir una opción para buscar integraciones por identificador.

44. La interfaz gráfica debe permitir ordenar las integraciones por latencia.

45. La interfaz gráfica debe permitir seleccionar una integración y ejecutar operaciones.

46. Los resultados de las operaciones deben mostrarse dentro de la ventana.

47. La interfaz gráfica debe mostrar mensajes cuando se ingresen datos incorrectos.

48. El programa debe manejar las entradas incorrectas sin terminar inesperadamente.

49. El sistema debe utilizar encapsulamiento para proteger los atributos de las clases.

50. El programa debe sobrescribir métodos como toString() y equals() cuando corresponda.

51. El código debe incluir comentarios que expliquen las clases y los métodos importantes.

52. La implementación debe coincidir con las clases, interfaces y relaciones representadas en el UML.

Consideraciones del diseño

El sistema utilizará una clase general llamada IntegracionAPI, que permitirá compartir los datos comunes de las integraciones. Las clases particulares representarán los diferentes tipos de servicios externos y tendrán sus propios comportamientos. Las interfaces Transaccional, Documentable y Auditable permitirán definir qué operaciones puede realizar cada integración sin obligar a todas las clases a implementar los mismos métodos. Para administrar las integraciones se utilizará una sola colección polimórfica. Esto permitirá guardar objetos de diferentes categorías y trabajar con ellos mediante sus comportamientos comunes. También se utilizará Comparable para ordenar las integraciones según su latencia promedio. La aplicación se organizará mediante MVC para separar la información, la lógica de funcionamiento y la interacción con el usuario. Además, utilizaremos Apache para crear una interfaz gráfica que permita realizar las operaciones mediante botones, tablas y campos de texto.

3. Clases e interfaces necesarias y sus responsabilidades

Para desarrollar el programa utilizaremos diferentes clases e interfaces que permitan administrar las conexiones externas de una plataforma de comercio electrónico. El sistema estará organizado mediante el patrón Modelo-Vista-Controlador (MVC), separando las operaciones de las integraciones, la interacción con el usuario y el control de las funcionalidades.

Interfaz Transaccional

La interfaz Transaccional representa las operaciones financieras que pueden realizar algunas integraciones externas. Su responsabilidad es establecer los métodos necesarios para procesar cobros y reintegros de dinero. Las clases que implementen esta interfaz deberán proporcionar su propia forma de realizar estas operaciones. Esto permite que el sistema trabaje con diferentes proveedores de pagos utilizando los mismos métodos, sin depender de una implementación específica. Por ejemplo, una integración de tarjetas y otra de billeteras digitales pueden implementar Transaccional, aunque internamente procesen sus operaciones de manera diferente.

Interfaz Documentable

La interfaz Documentable representa las integraciones que pueden generar documentos. Su responsabilidad es definir un comportamiento común para solicitar documentos a los proveedores externos. Esta interfaz puede utilizarse en servicios de facturación electrónica y logística, ya que ambos pueden necesitar generar documentos, aunque sean de diferentes tipos. Por ejemplo, una integración FEL puede generar una factura electrónica, mientras que una integración logística puede generar una guía de envío. Cada clase implementará el método de acuerdo con su función.

Interfaz Auditable

La interfaz Auditable representa las integraciones que permiten registrar o consultar información relacionada con las operaciones realizadas. Su responsabilidad es definir un comportamiento común para obtener información de auditoría. Esto permite que diferentes integraciones puedan proporcionar información sobre sus operaciones sin necesidad de que el sistema principal conozca los detalles internos de cada proveedor. No todas las integraciones necesitan implementar esta interfaz, porque algunas solamente permiten consultas o realizan funciones específicas.

Clase IntegracionAPI

La clase IntegracionAPI representa la información común de las conexiones externas del sistema. Se encarga de guardar el identificador de la integración, nombre del proveedor, estado de conexión y tiempo de latencia promedio. También permite consultar la información general y los parámetros de configuración de cada integración. Esta clase será la base de las diferentes categorías de APIs, permitiendo compartir atributos y métodos para evitar repetir código. Además, implementará la interfaz Comparable, que permitirá comparar las integraciones según su latencia promedio y ordenarlas de menor a mayor. Gracias a esta clase, el sistema podrá almacenar diferentes tipos de integraciones dentro de una sola colección polimórfica.

Clase PagosAPI

La clase APIPagos representa las integraciones relacionadas con los servicios de pago de una plataforma de comercio electrónico. Su responsabilidad es permitir la simulación de operaciones financieras, como cobrar y reintegrar dinero. Esta clase hereda de IntegracionAPI e implementa la interfaz Transaccional, porque necesita realizar operaciones financieras. También puede implementar Auditable cuando se requiera consultar información relacionada con las transacciones. Por ejemplo, el sistema puede utilizar esta clase para representar proveedores de tarjetas de crédito, transferencias bancarias o billeteras digitales. Cada objeto tendrá su propia configuración, identificador y latencia.

Clase LogisticaAPI

La clase APILogistica representa las integraciones relacionadas con el transporte y la entrega de productos. Su responsabilidad es permitir consultas relacionadas con los envíos y generar documentos cuando corresponda. Esta clase hereda de IntegracionAPI y puede implementar Documentable para representar la generación de guías de envío. Una característica importante es que no necesita implementar Transaccional, porque los proveedores logísticos no realizan reintegros financieros dentro de nuestro sistema. Si el usuario intenta realizar un reintegro utilizando una integración logística, el programa debe

ignorar la operación y mostrar un mensaje, sin detener su funcionamiento. Esto corresponde a una de las situaciones principales planteadas en el ejercicio.

Clase FelAPI

La clase APIFacturacion representa las integraciones utilizadas para generar documentos de facturación electrónica. Su responsabilidad es simular la comunicación con proveedores de Factura Electrónica en Línea (FEL). Esta clase hereda de IntegracionAPI e implementa Documentable, porque necesita generar documentos relacionados con la facturación. También puede utilizar Auditable para consultar información de las operaciones realizadas. La clase no necesita implementar operaciones financieras que no correspondan a su función.

Clase ConsultasAPI

La clase APIConsultas representa las integraciones que solamente permiten consultar información. Su responsabilidad es obtener datos relacionados con el estado de un servicio, una cotización o alguna operación externa. Esta clase hereda de IntegracionAPI y utiliza los comportamientos comunes definidos para las integraciones. No necesita implementar Transaccional si no permite cobros ni reintegros. Esto permite demostrar que las diferentes integraciones pueden compartir una clase padre sin tener exactamente las mismas capacidades.

Clase CatalogoIntegraciones

La clase CatalogoIntegraciones se encarga de administrar las conexiones externas registradas en el sistema. Su responsabilidad principal es guardar todas las integraciones dentro de una sola colección polimórfica de objetos IntegracionAPI. También permite agregar integraciones, consultar las registradas, buscar una API por identificador y obtener el catálogo ordenado por latencia. Para ordenar las integraciones se utilizará Comparable, implementado en IntegracionAPI. Esta clase también permitirá trabajar con nuevas integraciones sin tener que modificar la estructura principal del catálogo. El objetivo es que el sistema pueda incorporar nuevos proveedores manteniendo las mismas operaciones generales.

Clase VistaConsola

La clase VistaConsola representa la interacción del programa mediante la terminal. Su responsabilidad es mostrar las opciones disponibles, recibir los datos ingresados por el usuario y presentar los resultados de las operaciones. Esta clase permitirá listar integraciones, buscar por identificador, ordenar por latencia y ejecutar las operaciones disponibles. No realizará directamente los cálculos ni modificará los datos internos de las integraciones. Su función será comunicarse con el controlador para solicitar las operaciones correspondientes.

Clase VentanaPrincipal

La clase VentanaPrincipal representa la interfaz gráfica del programa, desarrollada utilizando Java Swing. Su responsabilidad es permitir que el usuario interactúe con el sistema mediante una ventana con botones, campos de texto, una tabla y un área de resultados. La ventana mostrará las integraciones registradas, sus identificadores, proveedores, tipos, estados de conexión y latencias. También permitirá buscar una integración por ID, ordenar el catálogo por latencia y seleccionar operaciones como consultar, cobrar, reintegrar, generar documentos y auditar. Los resultados se mostrarán dentro de la misma ventana para

facilitar la interacción. Esta clase no realizará directamente las operaciones de las APIs, sino que utilizará los métodos del controlador.

Clase ControladorIntegraciones

La clase ControladorIntegraciones se encarga de comunicar las vistas con el modelo. Su responsabilidad es recibir las solicitudes del usuario y utilizar las clases correspondientes para realizar las operaciones. Por ejemplo, cuando el usuario presiona el botón para buscar una API, el controlador solicita al catálogo que encuentre la integración correspondiente. Cuando se selecciona una operación, el controlador coordina su ejecución y devuelve un mensaje con el resultado. También debe comprobar si una integración admite la operación solicitada, evitando ejecutar comportamientos que no le corresponden. De esta manera, el controlador permite utilizar el mismo modelo desde la consola y desde la interfaz gráfica.

Clase Main

La clase Main contiene el método principal del programa. Su responsabilidad es crear los objetos necesarios para iniciar el sistema y cargar las integraciones iniciales. Al comenzar la ejecución se registrarán al menos diez conexiones de diferentes categorías, como pagos, logística, facturación y consultas. Después se crearán el controlador y la vista correspondiente para permitir que el usuario interactúe con el programa. Esta clase no realizará directamente los cálculos ni las operaciones de las integraciones, porque esas responsabilidades corresponden a las clases del modelo y al controlador.

Organización mediante MVC

El programa estará dividido en tres partes principales.

Modelo: estará formado por IntegracionAPI, APIPagos, APILogistica, APIFacturacion, APIConsultas, CatalogoIntegraciones y las interfaces Transaccional, Documentable y Auditable. Estas clases e interfaces se encargarán de representar las integraciones y sus comportamientos.

Vista: estará formada por VistaConsola y VentanaPrincipal. Ambas permitirán interactuar con el sistema, una mediante consola y otra mediante interfaz gráfica.

Controlador: estará formado por ControladorIntegraciones, que coordinará las solicitudes entre las vistas y el modelo.

La clase Main será responsable de iniciar el programa y crear los objetos principales.

Justificación del uso de interfaces

Utilizaremos interfaces porque las integraciones externas pueden realizar operaciones diferentes. Por ejemplo, una integración de pagos puede procesar cobros y reintegros, mientras que una integración logística puede consultar envíos o generar guías. Si todas las operaciones estuvieran obligatoriamente dentro de la clase padre, algunas integraciones tendrían métodos que no necesitan. Con las interfaces, cada categoría implementa solamente los comportamientos que le corresponden. Esto permite aprovechar el polimorfismo y facilita agregar nuevos proveedores sin modificar constantemente la lógica principal del sistema. Además, el uso de Comparable permite ordenar las integraciones de manera uniforme utilizando su latencia promedio.

Atributos, tipos, visibilidad y propósito

Los atributos de las clases serán privados (private) para mantener el encapsulamiento. Esto permite proteger la información de los objetos y acceder a ella mediante métodos públicos cuando sea necesario.

Interfaz Transaccional

La interfaz Transaccional no necesita atributos de instancia porque su función es definir los métodos que deben implementar las clases capaces de realizar operaciones financieras. Los datos necesarios para procesar cobros y reintegros serán administrados por las clases que implementen esta interfaz.

Interfaz Documentable

La interfaz Documentable no necesita atributos de instancia porque solamente establece el comportamiento que deben cumplir las integraciones capaces de generar documentos. Cada clase que implemente esta interfaz utilizará sus propios atributos para generar la información correspondiente.

Interfaz Auditable

La interfaz Auditable no necesita atributos de instancia porque define un contrato para las integraciones que permiten realizar operaciones de auditoría. Los datos utilizados para estas operaciones dependerán de la clase que implemente la interfaz.

Clase IntegracionAPI

id (String): guarda el identificador único de cada integración. Se utiliza para buscar una API específica dentro del catálogo.

proveedor (String): guarda el nombre de la empresa o servicio externo al que pertenece la integración.

conectada (boolean): indica si la integración se encuentra conectada o desconectada. Este atributo permite mostrar su estado de conexión al usuario.

latencia (int): guarda el tiempo promedio de respuesta de la integración en milisegundos. Se utiliza para ordenar el catálogo e identificar cuáles servicios tienen mayores tiempos de respuesta.

Clase APIPagos

Esta clase utilizará los atributos heredados de IntegracionAPI y tendrá información particular relacionada con las operaciones financieras.

metodoPago (String): indica el tipo de servicio de pago que representa la integración, como tarjeta de crédito, transferencia bancaria o billetera digital.

totalCobrado (double): guarda el dinero acumulado mediante las operaciones de cobro simuladas.

totalReintegrado (double): guarda el dinero acumulado mediante las operaciones de reintegro simuladas.

Clase APILogistica

Esta clase utilizará los atributos comunes de IntegracionAPI y tendrá información relacionada con los servicios de envío.

tipoEnvio (String): guarda el tipo de servicio logístico que representa la integración, como entrega estándar, exprés o última milla.

zonaCobertura (String): guarda la zona donde el proveedor ofrece sus servicios de entrega.

Clase APIFacturacion

Esta clase utilizará los atributos heredados de IntegracionAPI y tendrá información particular de los servicios de facturación electrónica.

certificador (String): guarda el nombre del certificador de Factura Electrónica en Línea utilizado por la integración.

documentosGenerados (int): guarda la cantidad de documentos que se han generado durante la simulación.

Clase APIConsultas

Esta clase utilizará los atributos comunes de IntegracionAPI y tendrá información particular de los servicios de consulta.

tipoConsulta (String): guarda el tipo de información que permite consultar la integración, como estados de pedidos o cotizaciones.

consultasRealizadas (int): guarda la cantidad de consultas ejecutadas durante el funcionamiento del programa.

Clase CatalogoIntegraciones

integraciones (ArrayList de IntegracionAPI): almacena todas las integraciones registradas en una única colección polimórfica. Permite guardar objetos de pagos, logística, facturación y consultas.

Esta colección será utilizada para listar, buscar y ordenar las integraciones. No será necesario crear una lista diferente para cada categoría, porque el sistema utilizará la clase padre IntegracionAPI para trabajar con todos los objetos. Esto es importante porque la rúbrica exige utilizar una sola estructura polimórfica para almacenar las integraciones.

Clase VistaConsola

teclado (Scanner): permite leer las opciones y los datos ingresados por el usuario desde la consola.

controlador (ControladorIntegraciones): guarda una referencia al controlador para solicitar las operaciones seleccionadas mediante el menú.

Clase VentanaPrincipal

Esta clase utilizará componentes de Java Swing para mostrar la interfaz gráfica.

controlador (ControladorIntegraciones): guarda la referencia al controlador que permite realizar las operaciones del sistema.

tablaModelo (DefaultTableModel): administra los datos que se muestran en la tabla de integraciones.

tabla (JTable): muestra las integraciones registradas, incluyendo su identificador, proveedor, tipo, conexión y latencia.

id (JTextField): permite ingresar el identificador de una integración para buscarla o seleccionarla.

referencia (JTextField): permite ingresar una referencia para identificar una operación.

monto (JTextField): permite ingresar la cantidad de dinero utilizada en las operaciones financieras simuladas.

operacion (JComboBox de String): permite seleccionar la operación que se desea ejecutar, como consultar, cobrar, reintegrar, generar documentos o auditar.

salida (JTextArea): muestra los mensajes y resultados de las operaciones realizadas.

resumen (JLabel): muestra información general sobre la cantidad de integraciones registradas y sus estados de conexión.

Clase ControladorIntegraciones

catalogo (CatalogoIntegraciones): guarda la referencia al objeto que administra las integraciones registradas. Este atributo permite que el controlador solicite búsquedas, ordenamientos y consultas sin almacenar otra colección de integraciones.

Clase Main

La clase Main no necesita atributos de instancia porque su responsabilidad será iniciar el programa. Dentro del método main() se crearán los objetos necesarios para cargar las integraciones, iniciar el controlador y mostrar la interfaz gráfica.

Valores iniciales de los atributos

- Cuando se crea una integración, su identificador, proveedor, estado de conexión y latencia se reciben mediante el constructor de IntegracionAPI.
- Las clases hijas utilizarán super() para inicializar los atributos comunes y después asignarán sus valores particulares.
- Los contadores utilizados para representar operaciones simuladas comenzarán en cero, salvo que se establezcan otros valores durante la carga inicial.
- Cuando se crea CatalogoIntegraciones, su colección comenzará vacía.
- Después, el programa registrará al menos diez integraciones de diferentes categorías. Algunas podrán comenzar conectadas y otras desconectadas para demostrar ambos estados.
- Cuando se crea VentanaPrincipal, se inicializarán los componentes de Java Swing y se cargarán los datos del catálogo en la tabla.
- La clase ControladorIntegraciones recibirá la referencia al catálogo mediante su constructor, permitiendo que las vistas utilicen sus métodos.

Métodos, parámetros, tipos de retorno y propósito

Interfaz Transaccional

La interfaz Transaccional establece los métodos que deben implementar las integraciones capaces de realizar operaciones financieras.

cobrar(double monto)*:* boolean: recibe una cantidad de dinero y permite realizar un cobro. Devuelve true cuando la operación se realiza correctamente y false cuando no se puede completar.

reintegrar(double monto): boolean: recibe una cantidad de dinero y permite procesar un reintegro. Devuelve true cuando se realiza correctamente y false cuando la operación no puede completarse.

Interfaz Documentable

generarDocumento(String referencia): String: recibe una referencia que identifica la operación y devuelve una cadena con la información del documento generado.

Interfaz Auditable

La interfaz Auditable establece un comportamiento común para las integraciones que permiten consultar información de auditoría.

auditar(String referencia) : String: recibe una referencia de operación y devuelve información relacionada con su auditoría.

Clase IntegracionAPI

IntegracionAPI(String id, String proveedor, boolean conectada, int latencia): constructor protegido que recibe los datos comunes de una integración y los utiliza para inicializar sus atributos.

getId() : String: devuelve el identificador de la integración.

getProveedor() : String: devuelve el nombre del proveedor externo.

isConectada() : boolean: devuelve true si la integración está conectada y false si está desconectada.

getLatencia() : int: devuelve la latencia promedio de la integración en milisegundos.

getTipo() : String: método abstracto que devuelve el tipo de integración. Cada clase hija lo implementará según su categoría.

configuracion() : String: devuelve una cadena con los parámetros de configuración de la integración, como su identificador, proveedor, conexión y latencia.

consultar(String referencia) : String: recibe una referencia y devuelve el resultado de una consulta simulada.

compareTo(IntegracionAPI otra) : int: compara la latencia de la integración actual con la de otra integración. Devuelve un número negativo, cero o positivo según corresponda. Se utiliza para ordenar mediante Comparable.

toString() : String: devuelve la información general de la integración.

equals(Object obj) : boolean: compara dos integraciones utilizando su identificador para determinar si representan la misma conexión.

hashCode() : int: genera un valor numérico basado en el identificador, manteniendo la consistencia con equals().

Clase APIPagos

APIPagos(String id, String proveedor, boolean conectada, int latencia, String metodoPago): constructor que recibe los datos comunes y el método de pago utilizado por la integración.

getMetodoPago() : String: devuelve el método de pago representado por la integración.

getTipo() : String: sobrescribe el método de IntegracionAPI y devuelve la categoría de pagos.

cobrar(double monto) : boolean: recibe una cantidad de dinero y simula su cobro cuando la integración está conectada y el monto es válido.

reintegrar(double monto) : boolean: recibe una cantidad de dinero y simula un reintegro financiero.

auditar(String referencia) : String: devuelve información de auditoría relacionada con una operación de pago, cuando la clase implementa Auditable.

toString() : String: devuelve la información común y particular de la integración de pagos.

Clase APILogistica

APILogistica(String id, String proveedor, boolean conectada, int latencia, String tipoEnvio, String zonaCobertura): constructor que recibe los datos comunes y las características del proveedor logístico.

getTipoEnvio() : String: devuelve el tipo de servicio de envío.

getZonaCobertura() : String: devuelve la zona de cobertura.

getTipo() : String: devuelve la categoría logística.

consultar(String referencia) : String: sobrescribe el comportamiento de consulta para mostrar información relacionada con un envío.

generarDocumento(String referencia) : String: genera una representación de una guía de envío utilizando la interfaz Documentable.

toString() : String: devuelve la información general y particular de la integración logística.

Clase APIFacturacion

APIFacturacion(String id, String proveedor, boolean conectada, int latencia, String certificador): constructor que inicializa los datos comunes y el nombre del certificador.

getCertificador() : String: devuelve el nombre del certificador FEL.

getTipo() : String: devuelve la categoría de facturación electrónica.

generarDocumento(String referencia) : String: simula la generación de un documento de facturación electrónica.

auditar(String referencia) : String: permite consultar información relacionada con la auditoría de documentos, cuando esta interfaz está implementada.

toString() : String: devuelve la información general y particular de la integración de facturación.

Clase APIConsultas

APIConsultas(String id, String proveedor, boolean conectada, int latencia, String tipoConsulta): constructor que recibe los datos comunes y el tipo de consulta que permite realizar.

getTipoConsulta() : String: devuelve el tipo de información que puede consultar la integración.

getTipo() : String: devuelve la categoría de consultas.

consultar(String referencia) : String: recibe una referencia y devuelve un resultado simulado relacionado con el servicio.

toString() : String: devuelve la información general y particular de la integración.

Clase CatalogoIntegraciones

CatalogoIntegraciones(): constructor público que inicializa la colección polimórfica donde se almacenarán las integraciones.

agregar(IntegracionAPI api) : boolean: recibe una integración y la agrega al catálogo si su identificador no está repetido. Devuelve true cuando se registra correctamente.

buscar(String id) : IntegracionAPI: recibe un identificador y busca la integración correspondiente. Devuelve el objeto encontrado o null si no existe.

listar() : ArrayList de IntegracionAPI: devuelve las integraciones registradas para mostrarlas en la consola o interfaz gráfica.

ordenar() : ArrayList de IntegracionAPI: devuelve las integraciones ordenadas de menor a mayor latencia utilizando la implementación de Comparable.

resumen() : String: devuelve información general sobre la cantidad de integraciones y cuántas están conectadas o desconectadas.

Clase ControladorIntegraciones

ControladorIntegraciones(CatalogoIntegraciones catalogo): constructor público que recibe el catálogo y guarda su referencia para administrar las operaciones.

listar() : ArrayList de IntegracionAPI: solicita al catálogo las integraciones registradas y devuelve la colección para mostrarla.

buscar(String id) : IntegracionAPI: busca una integración mediante su identificador y devuelve el objeto encontrado.

ordenar() : ArrayList de IntegracionAPI: obtiene las integraciones ordenadas por latencia.

resumen() : String: devuelve la información general del catálogo, incluyendo la cantidad de conexiones.

ejecutar(String id, String operacion, String referencia, double monto) : String: recibe el identificador de una integración, la operación seleccionada, una referencia y un monto. Busca la integración y verifica si puede realizar la operación solicitada. Si es compatible, utiliza la interfaz correspondiente. Si no lo es, devuelve un mensaje indicando que la operación fue ignorada.

Clase VistaConsola

VistaConsola(ControladorIntegraciones controlador): constructor que recibe el controlador e inicializa los elementos necesarios para interactuar mediante consola.

mostrarMenu() : void: muestra las opciones disponibles del sistema.

iniciar() : void: mantiene el menú funcionando hasta que el usuario decida salir.

mostrarIntegraciones() : void: solicita las integraciones al controlador y muestra su información.

buscarIntegracion() : void: solicita un identificador, busca la integración y muestra sus parámetros de configuración.

ordenarIntegraciones() : void: obtiene el catálogo ordenado por latencia y muestra sus resultados.

ejecutarOperacion() : void: solicita la integración y la operación que se desea realizar. Después muestra el resultado recibido del controlador.

Clase VentanaPrincipal

VentanaPrincipal(ControladorIntegraciones controlador): constructor público que recibe el controlador y crea la ventana gráfica utilizando Java Swing. Inicializa los botones, campos de texto, tabla y área de resultados.

cargar(ArrayList de IntegracionAPI datos) : void: método privado que recibe una colección de integraciones y actualiza las filas de la tabla.

seleccionar(String clave) : void: método privado que busca un identificador dentro de la tabla y selecciona la fila correspondiente.

mostrar(String mensaje) : void: método privado que recibe un mensaje y lo presenta en el área de resultados.

Además, la clase utiliza eventos de los botones para solicitar operaciones al controlador.

Por ejemplo, al presionar el botón de ordenar, la ventana utiliza controlador.ordenar() y actualiza la tabla mediante cargar(). Cuando se presiona el botón de ejecutar, obtiene los datos de los campos de texto y solicita al controlador realizar la operación seleccionada.

Clase Main

main(String[] args) : void: método público y estático donde comienza la ejecución del programa. Crea el catálogo, registra las integraciones iniciales, crea el controlador y abre la interfaz correspondiente.

El programa comenzará con al menos diez integraciones de diferentes categorías.

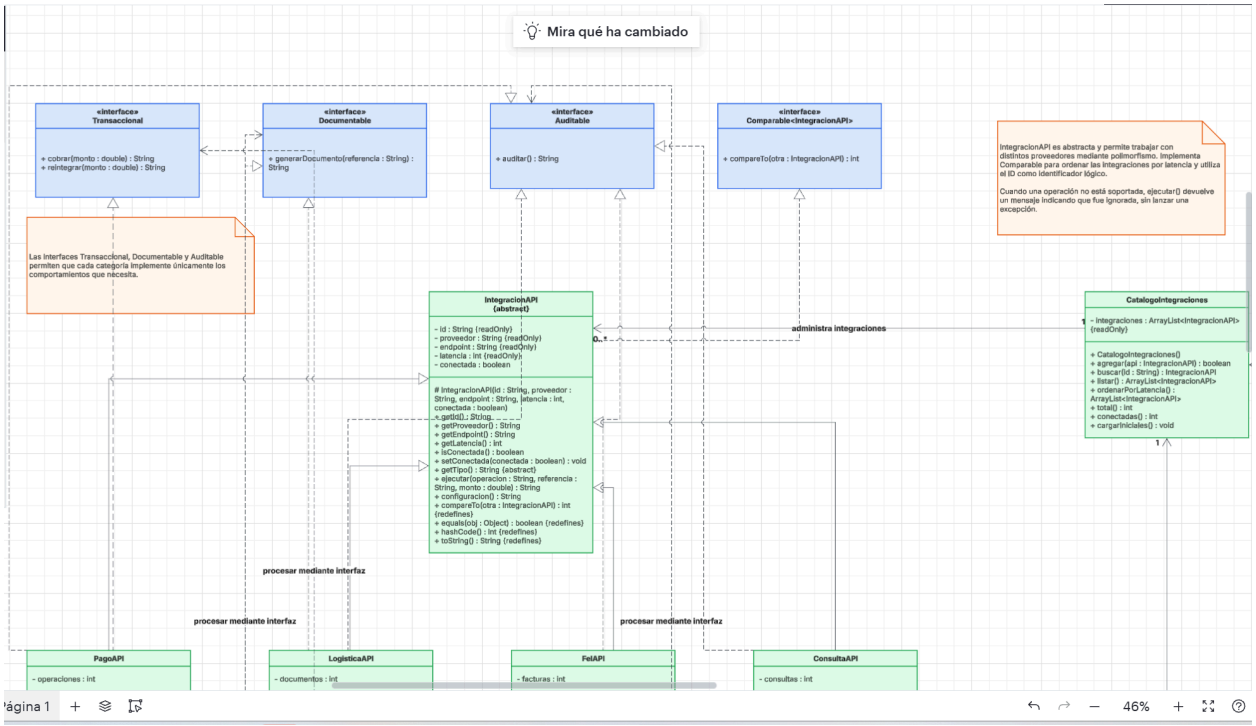
Uso de sobrecarga de métodos (Overloading)

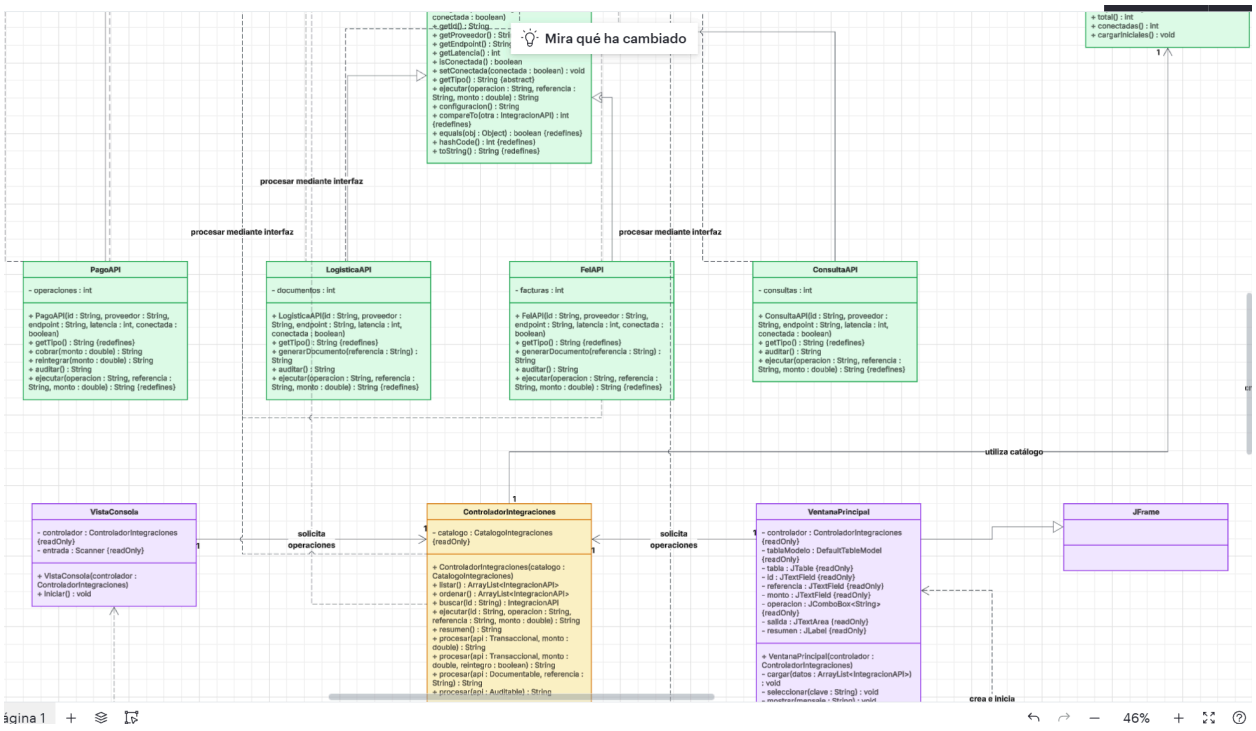
El uso de sobrecarga permite utilizar un mismo nombre de método con diferentes parámetros.

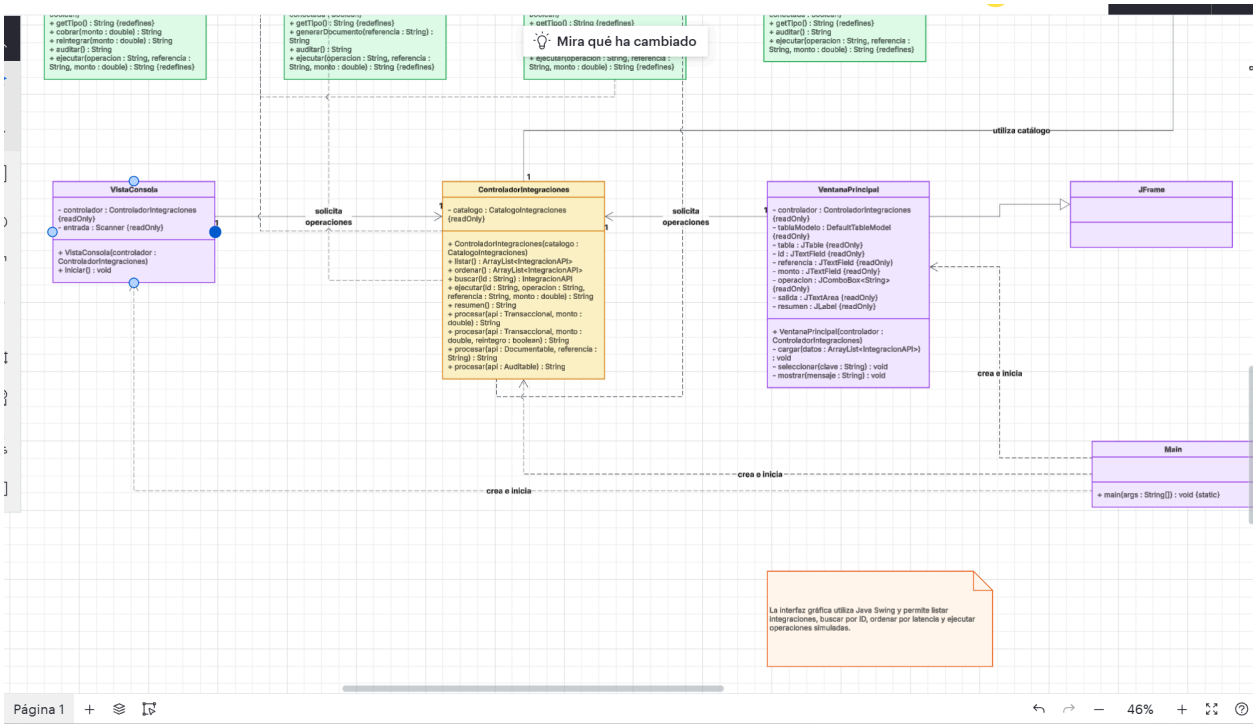
Uso de Comparable

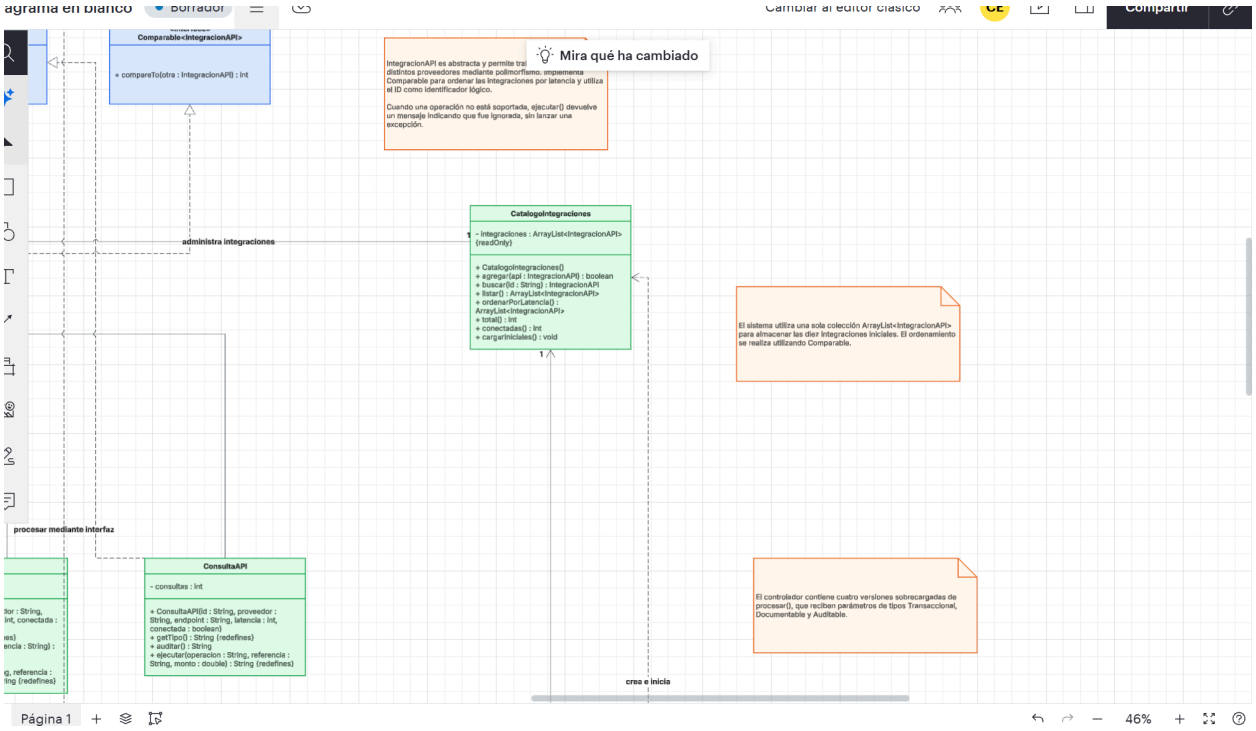
La clase IntegracionAPI implementará Comparable<IntegracionAPI> para permitir ordenar las integraciones por latencia. El método compareTo() comparará los milisegundos de dos integraciones. Si una integración tiene menor latencia, aparecerá antes en el catálogo ordenado.

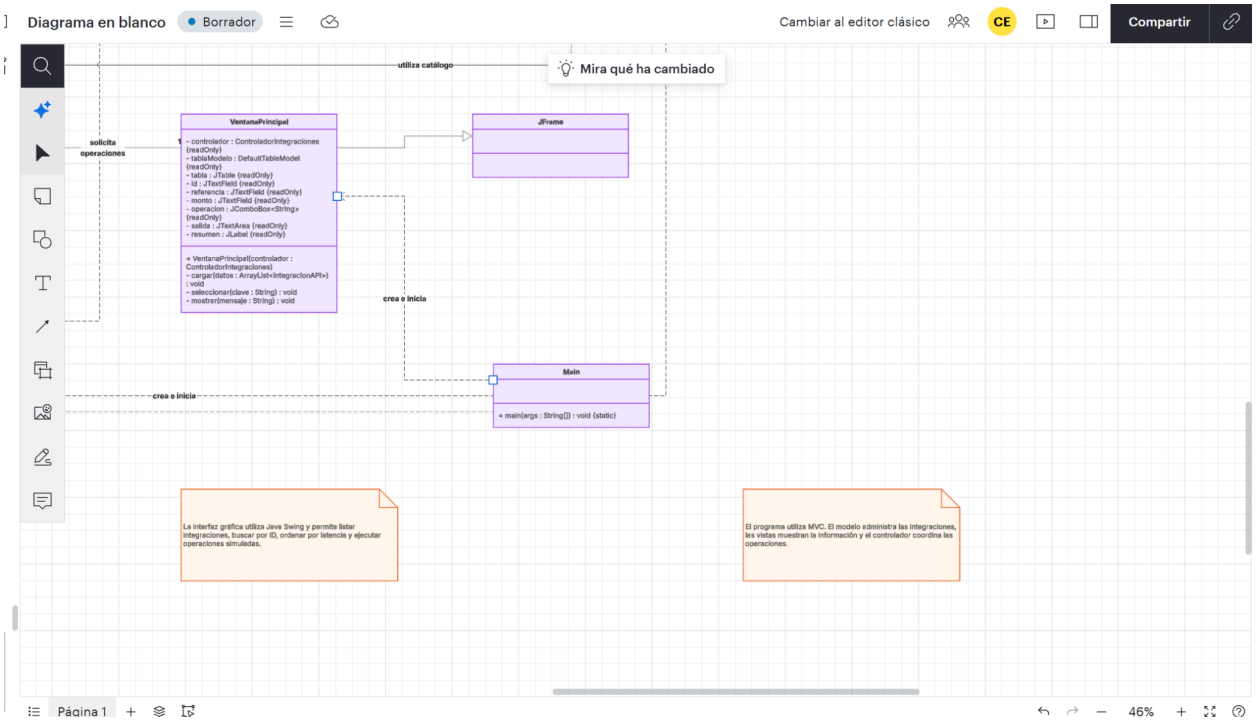
2. UML


Mira qué ha cambiado
«interface» Transaccional
+ cobrar(monto : double) : String
+ reintegrar(monto : double) : String
«interface» Documentable
+ generarDocumento(referencia : String) : String
«interface» Auditable
+ auditar() : String
«interface» Comparable<IntegracionAPI>
+ compareTo(otra : IntegracionAPI) : int
Las interfaces Transaccional, Documentable y Auditable permiten que cada categoría implemente únicamente los comportamientos que necesita.
IntegracionAPI es abstracta y permite trabajar con distintos proveedores mediante polimorfismo. Implementa Comparable para ordenar las integraciones por latencia y utiliza el ID como identificador lógico.
Cuando una operación no está soportada, ejecutar() devuelve un mensaje indicando que fue ignorada, sin lanzar una excepción.
IntegracionAPI (abstract)
- id : String {readOnly}
- proveedor : String {readOnly}
- endpoint : String {readOnly}
- latencia : int {readOnly}
- conectada : boolean
IntegracionAPI(id : String, proveedor : String, endpoint : String, latencia : int, conectada : boolean)
+ getId() : String
+ getProveedor() : String
+ getEndpoint() : String
+ getLatencia() : int
+ isConectada() : boolean
+ setConectada(conectada : boolean) : void
+ getTipo() : String {abstract}
+ ejecutar(operacion : String, referencia : String, monto : double) : String
+ configuracion() : String
+ compareTo(otra : IntegracionAPI) : int {redefines}
+ equals(obj : Object) : boolean {redefines}
+ hashCode() : int {redefines}
+ toString() : String {redefines}
administra integraciones
0..*
1
CatalogoIntegraciones
- integraciones : ArrayList<IntegracionAPI> {readOnly}
+ CatalogoIntegraciones()
+ agregar(api : IntegracionAPI) : boolean
+ buscar(id : String) : IntegracionAPI
+ listar() : ArrayList<IntegracionAPI>
+ ordenarPorLatencia() : ArrayList<IntegracionAPI>
+ total() : int
+ conectadas() : int
+ cargarIniciales() : void
procesar mediante interfaz
procesar mediante interfaz
procesar mediante interfaz
PagoAPI
- operaciones : int
LogisticaAPI
- documentos : int
FelAPI
- facturas : int
ConsultaAPI
- consultas : int
46%


Mira qué ha cambiado
procesar mediante interfaz
procesar mediante interfaz
procesar mediante interfaz
PagoAPI
- operaciones : int
+ PagoAPI(id : String, proveedor : String, endpoint : String, latencia : int, conectada : boolean)
+ getTipo() : String {redefines}
+ cobrar(monto : double) : String
+ reintegrar(monto : double) : String
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
LogisticaAPI
- documentos : int
+ LogisticaAPI(id : String, proveedor : String, endpoint : String, latencia : int, conectada : boolean)
+ getTipo() : String {redefines}
+ generarDocumento(referencia : String) : String
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
FelAPI
- facturas : int
+ FelAPI(id : String, proveedor : String, endpoint : String, latencia : int, conectada : boolean)
+ getTipo() : String {redefines}
+ generarDocumento(referencia : String) : String
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
ConsultaAPI
- consultas : int
+ ConsultaAPI(id : String, proveedor : String, endpoint : String, latencia : int, conectada : boolean)
+ getTipo() : String {redefines}
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
utiliza catálogo
VistaConsola
- controlador : ControladorIntegraciones {readOnly}
- entrada : Scanner {readOnly}
+ VistaConsola(controlador : ControladorIntegraciones)
+ iniciar() : void
solicita operaciones
ControladorIntegraciones
- catalogo : CatalogoIntegraciones {readOnly}
+ ControladorIntegraciones(catalogo : CatalogoIntegraciones)
+ listar() : ArrayList<IntegracionAPI>
+ ordenar() : ArrayList<IntegracionAPI>
+ buscar(id : String) : IntegracionAPI
+ ejecutar(id : String, operacion : String, referencia : String, monto : double) : String
+ resumen() : String
+ procesar(api : Transaccional, monto : double) : String
+ procesar(api : Transaccional, monto : double, reintegro : boolean) : String
+ procesar(api : Documentable, referencia : String) : String
+ procesar(api : Auditable) : String
solicita operaciones
VentanaPrincipal
- controlador : ControladorIntegraciones {readOnly}
- tablaModelo : DefaultTableModel {readOnly}
- tabla : JTable {readOnly}
- id : JTextField {readOnly}
- referencia : JTextField {readOnly}
- monto : JTextField {readOnly}
- operacion : JComboBox<String> {readOnly}
- salida : JTextArea {readOnly}
- resumen : JLabel {readOnly}
+ VentanaPrincipal(controlador : ControladorIntegraciones)
- cargar(datos : ArrayList<IntegracionAPI>) : void
- seleccionar(clave : String) : void
JFrame
crea e inicia
46%


Mira qué ha cambiado
+ getTipo() : String {redefines}
+ cobrar(monto : double) : String
+ reintegrar(monto : double) : String
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
+ getTipo() : String {redefines}
+ generarDocumento(referencia : String) : String
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
+ getTipo() : String {redefines}
+ getTipo() : String {redefines}
+ auditar() : String
+ ejecutar(operacion : String, referencia : String, monto : double) : String {redefines}
utiliza catálogo
VistaConsola
- controlador : ControladorIntegraciones {readOnly}
- entrada : Scanner {readOnly}
+ VistaConsola(controlador : ControladorIntegraciones)
+ iniciar() : void
solicita operaciones
ControladorIntegraciones
- catalogo : CatalogoIntegraciones {readOnly}
+ ControladorIntegraciones(catalogo : CatalogoIntegraciones)
+ listar() : ArrayList<IntegracionAPI>
+ ordenar() : ArrayList<IntegracionAPI>
+ buscar(id : String) : IntegracionAPI
+ ejecutar(id : String, operacion : String, referencia : String, monto : double) : String
+ resumen() : String
+ procesar(api : Transaccional, monto : double) : String
+ procesar(api : Transaccional, monto : double, reintegro : boolean) : String
+ procesar(api : Documentable, referencia : String) : String
+ procesar(api : Auditable) : String
solicita operaciones
VentanaPrincipal
- controlador : ControladorIntegraciones {readOnly}
- tablaModelo : DefaultTableModel {readOnly}
- tabla : JTable {readOnly}
- id : JTextField {readOnly}
- referencia : JTextField {readOnly}
- monto : JTextField {readOnly}
- operacion : JComboBox<String> {readOnly}
- salida : JTextArea {readOnly}
- resumen : JLabel {readOnly}
+ VentanaPrincipal(controlador : ControladorIntegraciones)
- cargar(datos : ArrayList<IntegracionAPI>) : void
- seleccionar(clave : String) : void
- mostrar(mensaje : String) : void
JFrame
crea e inicia
Main
+ main(args : String[]) : void {static}
crea e inicia
crea e inicia
La interfaz gráfica utiliza Java Swing y permite listar integraciones, buscar por ID, ordenar por latencia y ejecutar operaciones simuladas.
Página 1
46%


Comparable<IntegracionAPI>
+ compareTo(otra : IntegracionAPI) : int
Mira qué ha cambiado
IntegracionAPI es abstracta y permite tratar
distintos proveedores mediante polimorfismo, implementa
Comparable para ordenar las integraciones por latencia y utiliza
el ID como identificador lógico.
Cuando una operación no está soportada, ejecutar() devuelve
un mensaje indicando que fue ignorada, sin lanzar una
excepción.
administra integraciones
CatalogoIntegraciones
- integraciones : ArrayList<IntegracionAPI>
{readOnly}
+ CatalogoIntegraciones()
+ agregar(api : IntegracionAPI) : boolean
+ buscar(id : String) : IntegracionAPI
+ listar() : ArrayList<IntegracionAPI>
+ ordenarPorLatencia() :
ArrayList<IntegracionAPI>
+ total() : int
+ conectadas() : int
+ cargarIniciales() : void
El sistema utiliza una sola colección ArrayList<IntegracionAPI>
para almacenar las diez integraciones iniciales. El ordenamiento
se realiza utilizando Comparable.
procesar mediante interfaz
ConsultaAPI
- consultas : int
+ ConsultaAPI(id : String, proveedor :
String, endpoint : String, latencia : int,
conectada : boolean)
+ getTipo() : String {redefines}
+ auditar() : String
+ ejecutar(operacion : String, referencia :
String, monto : double) : String {redefines}
El controlador contiene cuatro versiones sobrecargadas de
procesar(), que reciben parámetros de tipos Transaccional,
Documentable y Auditable.
crea e inicia
Página 1
46%


Diagrama en blanco
Borrador
Cambiar al editor clásico
CE
Compartir
Mira qué ha cambiado
utiliza catálogo
solicita
operaciones
VentanaPrincipal
- controlador : ControladorIntegraciones
{readOnly}
- tablaModelo : DefaultTableModel
{readOnly}
- tabla : JTable {readOnly}
- id : JTextField {readOnly}
- referencia : JTextField {readOnly}
- monto : JTextField {readOnly}
- operacion : JComboBox<String>
{readOnly}
- salida : JTextArea {readOnly}
- resumen : JLabel {readOnly}
+ VentanaPrincipal(controlador :
ControladorIntegraciones)
- cargar(datos : ArrayList<IntegracionAPI>)
: void
- seleccionar(clave : String) : void
- mostrar(mensaje : String) : void
JFrame
crea e inicia
Main
+ main(args : String[]) : void {static}
crea e inicia
La interfaz gráfica utiliza Java Swing y permite listar
integraciones, buscar por ID, ordenar por latencia y ejecutar
operaciones simuladas.
El programa utiliza MVC. El modelo administra las integraciones,
las vistas muestran la información y el controlador coordina las
operaciones.
Página 1
46%